This problem becomes even more apparent when models receive updates. For instance, if OpenAI improves GPT coding capabilities, all your mocked coding responses might become outdated.

The core question then becomes, what are you truly testing? Are you assessing the model's performance or the system's behavior based on a fixed set of predefined responses? Mocking is useful for testing system logic and error handling, but it falls short when evaluating tasks that rely on the model's evolving capabilities. This brings us to the following topic of discussion.

#### WHAT TO MOCK VS. WHAT NOT TO MOCK

Deciding what to mock in testing LLM applications is a debatable topic. Mock components are best used for simpler, predictable tasks such as input sanitization, response processing, error handling, and basic flow control, as these do not require the complexity of real model interactions and allow for faster testing cycles. However, tasks like complex sentiment analysis, performance characteristics, and model reliability should not be mocked, as they require real model outputs for accurate validation. Additionally, when working with external LLM providers, testing can become more challenging since you lack control over their performance. Therefore, you need to find a balance when utilizing mocking for testing LLM responses.

#### CAVEAT: MOCKING EXPOSES IMPLEMENTATION DETAILS

Mocking LLM responses often requires you as a developer to make assumptions about the internal structure and behavior of the LLM system, which can lead to tightly coupled and brittle tests. When you create mocks, you essentially encode specific implementation details about how the LLM service works, rather than testing the actual behavior your application needs. Consider this example:

```
const mockLLMResponse = {
  choices: [{
    text: "Hello world",
    finish_reason: "stop",
    logprobs: {
      tokens: ["Hello", "world"],
      token_logprobs: [-0.22, -0.2]
    },
    model_version: "gpt-3.5-turbo"
  }],
};
const simplifiedMockResponse = {
  text: "Hello world",
  isComplete: true
};
```

The level of detail in the first `mockLLMResponse` can lead to tests that are overly reliant on the specific structure of the mocked response, making them fragile and difficult to maintain if the real model's output format changes. The `simplifiedMockResponse`, on the other hand, strips away unnecessary details and focuses on just the essential